Below are **30 advanced Java Stream API coding interview problems** commonly asked in companies like Google, Amazon, Meta, and other FAANG-level interviews.
The Stream API was introduced in **Java SE 8** and these problems test **deep understanding of streams, collectors, grouping, reduction, and functional programming**.

---

1. Find the Second Highest Number

```
List<Integer> list = List.of(5,9,3,7,8);

Integer second =
list.stream()
   .sorted(Comparator.reverseOrder())
   .skip(1)
   .findFirst()
   .orElse(null);

System.out.println(second);
```

---

2. Find the Third Highest Number

```
Integer third =
list.stream()
   .sorted(Comparator.reverseOrder())
   .skip(2)
   .findFirst()
   .orElse(null);
```

---

3. Find the First Non-Repeated Character

```
String str = "programming";

Character result =
str.chars()
  .mapToObj(c -> (char)c)
  .collect(Collectors.groupingBy(
      c -> c,
      LinkedHashMap::new,
      Collectors.counting()
  ))
  .entrySet()
  .stream()
  .filter(e -> e.getValue() == 1)
  .map(Map.Entry::getKey)
  .findFirst()
  .orElse(null);
```

---

4. Find the First Repeated Character

```
Character result =
str.chars()
  .mapToObj(c -> (char)c)
  .collect(Collectors.groupingBy(
      c -> c,
      LinkedHashMap::new,
      Collectors.counting()
  ))
  .entrySet()
  .stream()
  .filter(e -> e.getValue() > 1)
  .map(Map.Entry::getKey)
  .findFirst()
  .orElse(null);
```

---

5. Count Frequency of Each Character

```
Map<Character,Long> freq =
str.chars()
  .mapToObj(c -> (char)c)
  .collect(Collectors.groupingBy(
       c -> c,
       Collectors.counting()
  ));
```

---

6. Find Duplicate Numbers

```
Set<Integer> seen = new HashSet<>();

List<Integer> duplicates =
list.stream()
   .filter(n -> !seen.add(n))
   .toList();
```

---

7. Find Unique Elements

```
list.stream()
   .collect(Collectors.groupingBy(
       n -> n,
       Collectors.counting()
   ))
   .entrySet()
```

```
   .stream()
   .filter(e -> e.getValue()==1)
   .map(Map.Entry::getKey)
   .forEach(System.out::println);
```

---

8. Find Longest String

```
names.stream()
    .max(Comparator.comparing(String::length))
    .orElse(null);
```

---

9. Find Shortest String

```
names.stream()
    .min(Comparator.comparing(String::length))
    .orElse(null);
```

---

10. Convert List to Map

```
Map<Integer,String> map =
list.stream()
   .collect(Collectors.toMap(
      n -> n,
      n -> "Value" + n
   ));
```

---

11. Sort Map by Value

```
map.entrySet()
  .stream()
  .sorted(Map.Entry.comparingByValue())
  .forEach(System.out::println);
```

---

12. Group Strings by Length

```
Map<Integer,List<String>> result =
names.stream()
    .collect(Collectors.groupingBy(
      String::length
    ));
```

---

13. Partition Numbers into Even and Odd

```
Map<Boolean,List<Integer>> map =
list.stream()
   .collect(Collectors.partitioningBy(
      n -> n % 2 == 0
   ));
```

---

14. Find Top 3 Highest Numbers

```
list.stream()
   .sorted(Comparator.reverseOrder())
   .limit(3)
   .forEach(System.out::println);
```

---

15. Flatten Nested List

```
List<List<Integer>> nested =
List.of(List.of(1,2),List.of(3,4));

nested.stream()
     .flatMap(Collection::stream)
     .forEach(System.out::println);
```

---

16. Sum of Squares

```
int sum =
list.stream()
   .map(n -> n*n)
   .reduce(0,Integer::sum);
```

---

17. Find Average

```
double avg =
list.stream()
   .mapToInt(Integer::intValue)
   .average()
   .orElse(0);
```

---

18. Count Words in Sentence

```
String sentence = "Java stream API interview questions";

long count =
Arrays.stream(sentence.split(" "))
     .count();
```

---

19. Find Longest Word

```
Arrays.stream(sentence.split(" "))
    .max(Comparator.comparing(String::length))
    .orElse(null);
```

---

20. Remove Null Values

```
list.stream()
   .filter(Objects::nonNull)
   .toList();
```

---

21. Merge Two Lists

```
Stream.concat(list1.stream(), list2.stream())
    .toList();
```

---

22. Check If List Contains Duplicate

```
boolean result =
list.size() !=
list.stream().distinct().count();
```

---

23. Convert List to List

```
list.stream()
   .map(String::valueOf)
   .toList();
```

---

24. Find Maximum Using Reduce

```
int max =
list.stream()
   .reduce(Integer.MIN_VALUE,
      (a,b)->a>b?a:b);
```

---

25. Join Strings

```
String result =
names.stream()
    .collect(Collectors.joining(", "));
```

---

26. Find Numbers Greater Than Average

```
double avg =
list.stream()
   .mapToInt(Integer::intValue)
   .average()
   .orElse(0);

list.stream()
   .filter(n -> n > avg)
   .forEach(System.out::println);
```

---

27. Find Most Frequent Element

```
Integer result =
list.stream()
   .collect(Collectors.groupingBy(
      n -> n,
      Collectors.counting()
   ))
   .entrySet()
   .stream()
   .max(Map.Entry.comparingByValue())
   .get()
   .getKey();
```

---

28. Sort Employees by Salary

```
employees.stream()
      .sorted(Comparator.comparing(
         Employee::getSalary
      ))
      .forEach(System.out::println);
```

---

29. Highest Salary Employee

```
employees.stream()
      .max(Comparator.comparing(
         Employee::getSalary
      ))
      .orElse(null);
```

---

30. Group Employees by Department

```
Map<String,List<Employee>> result =
employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment
        ));
```

Summary of Advanced Interview Problems

Category	Examples
Character problems	repeated / non-repeated
Array problems	second highest
String problems	longest word
Map operations	sorting map
Aggregation	sum, average
Employee problems	Grouping

If you want, I can also show **something even more useful for FAANG Java interviews**:

1 **100 extremely tricky Stream interview questions**
2 **Complete list of 80+ Stream API methods with explanations**
3 **Internal working of Stream API (Pipeline, Spliterator, Sink, Node, ForkJoinPool)** — which is **asked in senior Java architecture interviews**.

Below are **50 real Java coding interview problems solved using the Stream API** (introduced in **Java SE 8**).
These problems are commonly asked in **Java developer interviews** and demonstrate real-world Stream usage.

---

1. Remove Duplicate Elements

```
List<Integer> list = List.of(1,2,2,3,4,4,5);

List<Integer> result =
list.stream()
   .distinct()
   .toList();

System.out.println(result);
```

Output

```
[1,2,3,4,5]
```

---

2. Find Even Numbers

```
list.stream()
   .filter(n -> n % 2 == 0)
   .forEach(System.out::println);
```

---

3. Find Odd Numbers

```
list.stream()
   .filter(n -> n % 2 != 0)
   .forEach(System.out::println);
```

---

4. Square Each Number

```
list.stream()
   .map(n -> n * n)
   .forEach(System.out::println);
```

---

5. Sum of Numbers

```
int sum =
list.stream()
   .reduce(0,Integer::sum);
```

---

6. Average of Numbers

```
double avg =
list.stream()
   .mapToInt(Integer::intValue)
   .average()
   .orElse(0);
```

---

7. Find Maximum Number

```
int max =
list.stream()
   .max(Integer::compare)
   .get();
```

---

8. Find Minimum Number

```
int min =
list.stream()
   .min(Integer::compare)
   .get();
```

---

9. Count Elements

```
long count =
list.stream().count();
```

---

10. Sort Numbers

```
list.stream()
   .sorted()
   .forEach(System.out::println);
```

---

11. Reverse Sort

```
list.stream()
   .sorted(Comparator.reverseOrder())
   .forEach(System.out::println);
```

---

12. Find First Element

```
list.stream()
   .findFirst()
   .ifPresent(System.out::println);
```

---

13. Find Any Element

```
list.stream()
   .findAny()
   .ifPresent(System.out::println);
```

---

14. Check Any Match

```
boolean result =
list.stream()
   .anyMatch(n -> n > 10);
```

---

15. Check All Match

```
boolean result =
list.stream()
   .allMatch(n -> n > 0);
```

---

16. Check None Match

```
boolean result =
list.stream()
   .noneMatch(n -> n < 0);
```

---

17. Convert List to Set

```
Set<Integer> set =
list.stream()
   .collect(Collectors.toSet());
```

---

18. Convert List to Map

```
Map<Integer,Integer> map =
list.stream()
   .collect(Collectors.toMap(
      n -> n,
      n -> n*n
   ));
```

---

19. Group Numbers by Even/Odd

```
Map<Boolean,List<Integer>> map =
list.stream()
   .collect(Collectors.partitioningBy(
      n -> n % 2 == 0
   ));
```

---

20. Frequency of Elements

```
Map<Integer,Long> freq =
list.stream()
   .collect(Collectors.groupingBy(
      n -> n,
      Collectors.counting()
   ));
```

---

21. Remove Null Values

```
list.stream()
   .filter(Objects::nonNull)
   .forEach(System.out::println);
```

---

22. Find Duplicate Elements

```
Set<Integer> seen = new HashSet<>();

list.stream()
   .filter(n -> !seen.add(n))
   .forEach(System.out::println);
```

---

23. Convert Strings to Uppercase

```
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

---

24. Convert Strings to Lowercase

```
names.stream()
    .map(String::toLowerCase)
    .forEach(System.out::println);
```

---

25. Filter Strings Starting with A

```
names.stream()
    .filter(s -> s.startsWith("A"))
    .forEach(System.out::println);
```

---

26. Find Longest String

```
names.stream()
    .max(Comparator.comparing(String::length))
    .ifPresent(System.out::println);
```

---

27. Find Shortest String

```
names.stream()
    .min(Comparator.comparing(String::length))
    .ifPresent(System.out::println);
```

---

28. Join Strings

```
String result =
names.stream()
    .collect(Collectors.joining(", "));
```

---

29. Count Strings by Length

```
Map<Integer,Long> map =
names.stream()
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.counting()
    ));
```

---

30. Find Second Highest Number

```
list.stream()
   .sorted(Comparator.reverseOrder())
   .skip(1)
   .findFirst()
   .ifPresent(System.out::println);
```

---

31. Flatten Nested Lists

```
List<List<Integer>> nested =
List.of(List.of(1,2),List.of(3,4));

nested.stream()
     .flatMap(Collection::stream)
     .forEach(System.out::println);
```

---

32. Skip First Two Elements

```
list.stream()
   .skip(2)
   .forEach(System.out::println);
```

---

33. Limit Elements

```
list.stream()
   .limit(3)
   .forEach(System.out::println);
```

---

34. Find Distinct Sorted Values

```
list.stream()
   .distinct()
   .sorted()
   .forEach(System.out::println);
```

---

35. Convert Integer List to String List

```
List<String> result =
list.stream()
   .map(String::valueOf)
   .toList();
```

---

36. Calculate Product of Numbers

```
int product =
list.stream()
   .reduce(1,(a,b)->a*b);
```

---

37. Sum of Squares

```
int sum =
list.stream()
```

```
   .map(n -> n*n)
   .reduce(0,Integer::sum);
```

---

38. Find Numbers Greater than 10

```
list.stream()
   .filter(n -> n > 10)
   .forEach(System.out::println);
```

---

39. Count Even Numbers

```
long count =
list.stream()
   .filter(n -> n % 2 == 0)
   .count();
```

---

40. Check Duplicate Exists

```
boolean hasDuplicate =
list.stream()
   .distinct()
   .count() != list.size();
```

---

41. Find First Non-Repeated Character

```
String str = "java";

Character c =
str.chars()
  .mapToObj(x -> (char)x)
  .filter(ch -> str.indexOf(ch) == str.lastIndexOf(ch))
  .findFirst()
  .orElse(null);
```

---

42. Count Characters in String

```
Map<Character,Long> map =
str.chars()
  .mapToObj(c -> (char)c)
  .collect(Collectors.groupingBy(
      c -> c,
      Collectors.counting()
  ));
```

---

43. Convert Array to List

```
Integer[] arr = {1,2,3};

List<Integer> list =
Arrays.stream(arr)
    .toList();
```

---

44. Sort Strings by Length

```
names.stream()
   .sorted(Comparator.comparing(String::length))
   .forEach(System.out::println);
```

---

45. Partition Numbers

```
Map<Boolean,List<Integer>> map =
list.stream()
  .collect(Collectors.partitioningBy(
     n -> n > 5
  ));
```

---

46. Find Max Salary Employee

```
employees.stream()
      .max(Comparator.comparing(Employee::getSalary))
      .get();
```

---

47. Group Employees by Department

```
employees.stream()
      .collect(Collectors.groupingBy(
         Employee::getDepartment
      ));
```

---

48. Average Salary

```
double avg =
employees.stream()
      .collect(Collectors.averagingDouble(
         Employee::getSalary
      ));
```

---

49. Find Employee Names

```
employees.stream()
    .map(Employee::getName)
    .forEach(System.out::println);
```

---

50. Sort Employees by Salary

```
employees.stream()
    .sorted(Comparator.comparing(
        Employee::getSalary
    ))
    .forEach(System.out::println);
```

Summary of Stream Problems

Category	Examples
Filtering	even numbers, null removal
Mapping	square numbers
Reduction	sum, product
Aggregation	max, min
Grouping	groupingBy
Partitioning	partitioningBy
Transformation	Map
Flattening	flatMap

If you want, I can also show the **next advanced things that make you extremely strong in Java interviews**:

1 **30 advanced Stream interview coding questions asked in FAANG companies**
2 **Complete list of 80+ Stream API methods with explanation**
3 **Internal architecture of Stream API (Spliterator, Sink, Pipeline, ForkJoinPool)** — rarely explained but very important for senior Java interviews.